



Institut supérieur de
mécanique de Paris

Contribution to real-time SI engine combustion modeling to reproduce standard cycles

STG2 - Non Confidential

CMT – Clean Mobility & Thermofluids
Universitat Politècnica de València



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA

Thomas FOURNIER

September 2025 / January 2026

Supervisor : Mr J. Javier López

Abstract

During this internship, I was able to develop a numerical model for predicting the ignition of a mixture in SI an engine, in line with the VEMOD project conducted at the Universitat Politècnica de València.

We started by creating a basic 4D simulation tool. The purpose of this simulation was to establish a first simple model, considering 4 fixed parameters. We then proceeded to densify the model by finding a way to interpolate all the data acquired thanks to the simulation tool. This model, once interpolated, is the most important as it has to be as accurate as possible not to scale the errors to the next step. Finally, we were able to get a more complex model allowing the parameters to change with time.

Nevertheless, the developed model could still be improved, particularly with a denser dataset, which could lead to a more efficient interpolation method, perhaps using neural network. We could also get an overview of the models accuracy once the bench test is available, by comparing it to a real SI engine

Résumé

Durant ce stage, j'ai pu développer un modèle numérique de prédiction d'auto-allumage d'un mélange air/carburant pour des moteurs à allumage commandé, en lien avec le projet VEMOD mené par l'Universitat Politècnica de València

Nous avons commencé par créer un outil de simulation en 4 dimensions. L'objectif de cette simulation était d'établir un premier modèle simplifié, en considérant 4 paramètres fixes. Nous avons ensuite densifié le modèle en trouvant une méthode d'interpolation sur l'ensemble des données obtenues grâce à l'outil de simulation. Le modèle interpolé est le plus important, car il doit être le plus précis possible afin d'éviter de propager les erreurs à l'étape suivante. Enfin, nous avons cherché à établir un modèle plus complexe permettant l'évolution des paramètres avec le temps.

Cependant, le modèle développé peut encore être amélioré notamment grâce à un jeu de données plus dense, ce qui pourrait conduire à une interpolation plus efficace, peut être même basée sur des réseaux de neurones. Nous pourrions également évaluer la précision du modèle une fois les tests sur bancs d'essais possibles, en le comparant à un moteur réel disponible directement sur site.

Acknowledgements

I would like to express my sincere gratitude to everyone who contributed to the successful accomplishment of my internship and the preparation of this report.

First of all, I would like to thank ISAE–Supméca for providing me with the opportunity to benefit from this experience at Universitat Politècnica de València.

I am also very grateful to CMT - Clean Mobility & Thermofluids for welcoming me into its research environment and providing access to its facilities and resources.

Then, I would like to thank Mr. J. Javier López and Mr. Daniel Robayo Rueda for their guidance, their knowledge, and the trust they placed in me throughout this internship. Their expertise and advice were invaluable for the success of this work.

I also wish to express my gratitude to Mr. Erwan Barbot and Mr. Maël Rodier, for all the memorable time spent together that made this experience even more enjoyable.

Lastly, I would like to thank my loved ones for their constant support, patience, and encouragement throughout this journey.

Table of contents

Abstract	2
Résumé	2
Acknowledgements	3
Table of contents	4
1 Introduction	5
1.1 Host-Organization presentation	5
1.1.1 History	5
1.1.2 Working environment and Partners	5
1.2 Sustainable development	7
1.3 Societal responsibilities	7
2 Positioning of the problem	8
2.1 Fundamentals of internal combustion engines	8
2.2 VEMOD Project	9
2.3 HVO Project	9
2.4 Objectives Description	9
3 Executed missions	11
3.1 Simulation	11
3.1.1 Simulation Tools - Cantera	11
3.1.2 Method for detecting auto-ignition	12
3.1.3 Optimal method and refinement	13
3.1.4 Dataset	14
3.2 Interpolation	15
3.2.1 Radial Basis Function Interpolation	15
3.2.2 Gaussian Process Regression Interpolation	16
3.2.3 Neural Network	17
3.3 Results	18
3.3.1 Accuracy of interpolators	18
3.3.2 IDT for variable parameters	20
3.3.3 Verification on a real engine cycle	20
3.4 Post-processing of bench tests results	22
4 Conclusion	24
5 Appendix	28

1 Introduction

1.1 Host-Organization presentation

1.1.1 History

CMT – Clean Mobility & Thermofluids, the UPV - Universitat Politècnica de València University research institute, conducts fundamental and applied research for the automotive, maritime, energy and aerospace sectors. With more than 140 members forming a single research group, it is one of the world's largest teams in thermo fluid dynamics.



Figure 1: CMT's infrastructure

Since its creation in 1979, CMT has progressively oriented its research to processes based on thermofluids that contribute to reduction of pollution, improved efficiency, and decarbonization. The institute now applies its extensive expertise to new fields, including vehicle electrification and renewable fuels.

CMT also transfers its scientific results into high value technological solutions, fostering cross sector collaboration to support societal, economic, and industrial innovation.

1.1.2 Working environment and Partners

CMT is a massive facility that covers 12,100 square meters. The campus boasts cutting-edge technological resources, including 22 power plant test cells that play a major role in validating numerical models, as well as 25 experimental environments. With these facilities, CMT stands out as a leading research center in the mechanical and automotive fields.

With about 190 scientific books published, 940 journal articles, and more than 1,000 conference presentations, the university demonstrates its strong commitment to applied research. These publications and achievements highlight the commitment of CMT to sharing acquired knowledge. Table 1 displays data on doctoral theses and research projects.

Status	Research Projects	Doctoral theses
Completed	815	225
In progress	70	65

Table 1: Status of research projects and doctoral theses

To carry out all its projects, CMT receives funding primarily from industrial partners, followed by public institutions. In total, 25 million euros are allocated each year. Figure 2 below shows the distribution of CMT’s funds.

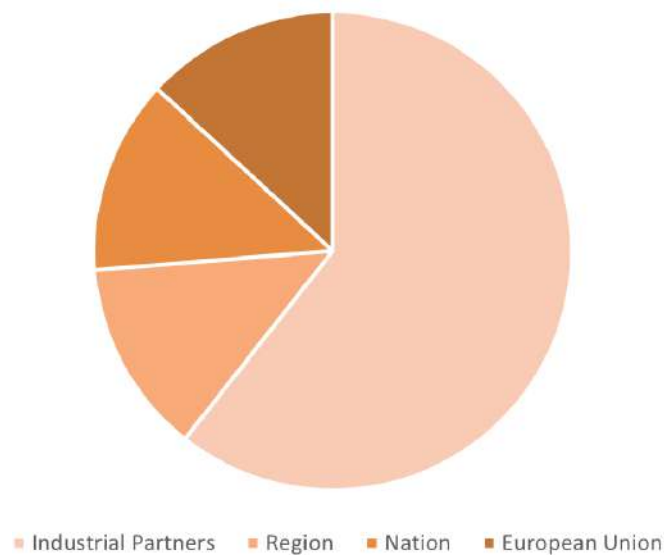


Figure 2: Distribution of CMT’s funds

The center is located on the UPV campus, which facilitates collaboration. Thanks to UPV’s reputation—ranked in the QS World University Ranking 2022 and featured in “Reuters Top 100: Europe’s Most Innovative Universities 2020”—CMT has the opportunity to develop its activities with international university research centers.

CMT works with numerous industrial partners, most of them in the automotive sector. It currently collaborates with car manufacturers such as Nissan, Renault, Volvo Powertrains and PSA, and has previously worked with Ford, Toyota, and BMW on diesel combustion, as well as Ferrari and Jaguar Land Rover on turbocharging and noise quality. Other partners include Aprilia (motorcycles), Guascor (marine engines), Repsol and Air Liquide (energy), and EMT – València, the city’s public transport company.

1.2 Sustainable development

The UPV is a Spanish public university deeply committed to sustainable development. It is part of the United Nations Global Compact, along with more than 14,000 companies and 4,000 non-business organizations working to reduce their environmental impact. As part of UPV, CMT conducts research projects in agreement with the university's sustainability goals.

CMT is actively engaged in research on internal combustion engines using zero-emission fuels. To support this work, the center is equipped with a cutting-edge emissions laboratory, which contains advanced systems to measure gaseous and particulate pollutants. These include analyzers for CO/CO_2 and NO/NO_x , as well as an engine exhaust particle spectrometer.

CMT recently went through a strategic rebranding, evolving from its former name, Motores Térmicos, to Clean Mobility & Thermofluids. This change reflects its wider perspective of sustainable development and its commitment to dealing with the challenges of clean mobility.

1.3 Societal responsibilities

Over and above its scientific and technological mission, CMT has important societal responsibilities as part of the UPV. These responsibilities reflect the commitment to education, inclusion, and collaboration with an extended community.

CMT helps prepare future engineers and researchers by providing internships, PhD programs, and mobility opportunities. These efforts ensure equal access to quality education and encourage international cooperation.

The center also contributes to initiatives such as Science Meets Regions, which promote dialogue between researchers, local authorities, and industry leaders. This engagement helps coordinate scientific innovation with societal needs and regional development.

Finally, thanks to its partnerships with leading companies in fields of automotive and energy industry, CMT supports innovation by enhancing the performance of local and international industries. These collaborations create economic value and opportunities for skilled employment.

By integrating these aspects into its activities, CMT shows that research excellence goes hand in hand with social responsibility, contributing to education, economic development, and ethical engagement with society.

2 Positioning of the problem

2.1 Fundamentals of internal combustion engines

Internal combustion engines, otherwise called an ICE, are engines that create motive power by burning fuel with air inside the engine itself [1]. It produces burned gases that are used to drive a piston. This process transforms chemical energy into kinetic energy.

In this section, we will focus on the principle of a 4-Stroke SI engine, as this is the type of engine we had access to at the CMT.

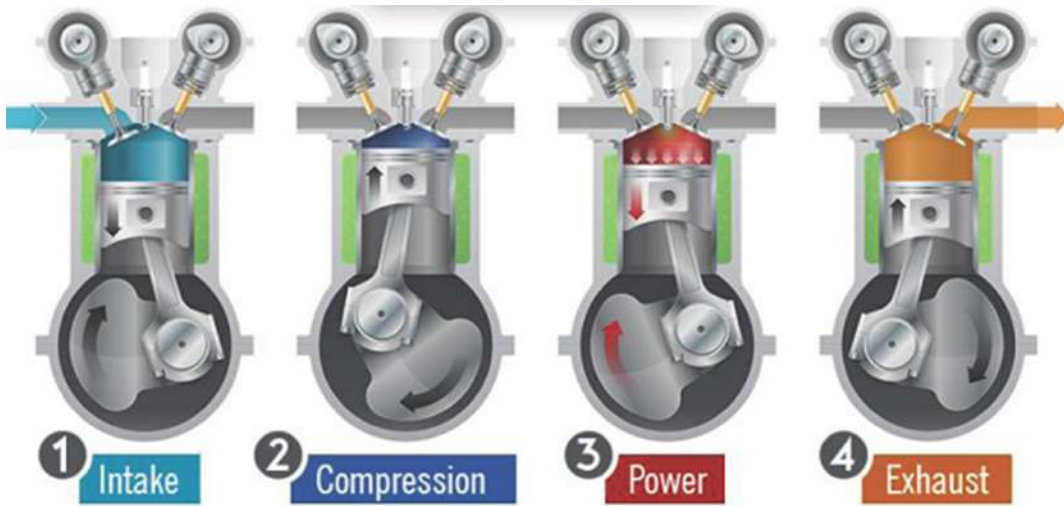


Figure 3: 4-Stroke SI engine cycle

- **Intake:** Otherwise called induction or suction. this stroke of the piston begins at Top Dead Center (TDC) and ends at Bottom Dead Center (BDC). During this stroke, the piston pulls an air/fuel mixture into the cylinder by producing a vacuum in the cylinder through its downward motion. Therefore, the intake valve must be in its open position to let the mixture go in.
- **Compression:** Unlike the intake stroke, this stroke of the piston begins at BDC and ends at TDC. In this stroke, the piston compresses the air-fuel mixture using a part of the mechanical work produced by the engine (below). To achieve this compression, both the intake and exhaust valves must be closed.
- **Combustion:** Also known as power or ignition. At this point, the crankshaft has completed a full 360° revolution. While the piston reached the TDC at the end of the compression stroke, the mixture is ignited by a spark plug. In compression ignition engines (CI), the mixture ignites by its own because of the heat generated by the compression. In both cases, the ignition produces mechanical work.
- **Exhaust:** Otherwise called exhaust outlet. Once again, during this stroke the piston returns from BDC to TDC. This time, the exhaust valve is open to expels the spent mixture through the exhaust port.

2.2 VEMOD Project

The Virtual Engine Model project (VEMOD) is the main project which I had to work in. At first, this project began in 2016 in association with Renault, but this collaboration ended in 2018. The research team working on the project decided to keep working on it in order to make this software an open-source software.

This software aims to create a model to reproduce standard cycles for ICE. Several works have already been done, but there remain some aspects that need to be studied, regarding the prediction of the behaviour of the engines.

While software such as OpenFOAM already exists, VEMOD does not focus on computational fluid dynamics (CFD) but chemical and physical kinetics modeling.

2.3 HVO Project

The main goal of the Hydrotreated Vegetable Oil (HVO) project is to analyze the sustainability of HVO as an alternative fuel for SI engines. Even if HVO has already been introduced to the market, we do not fully understand its behaviour. Some major aspects such as its combustion behaviour, engine performance, and long-term durability still require more detailed study. These studies are essential to determine whether or not HVO can be considered as a sustainable and reliable solution for future mobility.

Several experimental studies have already been carried out. Experimental work focuses on analyzing combustion efficiency, pollutant species, and compatibility with existing SI engines. Most of the tests are used to compare this fuel to diesel. Indeed, data are firstly acquired using diesel and then using HVO to achieve a comparison and assess its features.

Finally, the goal of this research is to know if HVO can be a reliable alternative to conventional fuels, ensuring compliance with future emission standards and supporting the decarbonization of the automotive field.

2.4 Objectives Description

During this internship, we aim at both VEMOD and HVO project. The main work aims to develop and validate a numerical model capable of predicting auto-ignition phenomena in spark-ignition engines [2], part of the VEMOD project. Auto-ignition, which occurs when the fuel-air mixture ignites spontaneously before or after the spark event, can significantly influence engine performance, efficiency, and durability. Understanding and predicting this behaviour is therefore critical for the development of alternative fuels.

Previous work has provided essential knowledge on ignition delay time. Indeed, we know how to measure the time between the ignition command and the actual spark ignition, and we understand

the consequences of auto-ignition depending on when it occurs during the cycle. However, the conditions under which the mixture self-ignites, and the corresponding ignition delay time (IDT), remain to be determined for the fuel under study.

The ignition delay time, denoted as τ , can be defined as the period from the onset of a given temperature and pressure of a combustible gas mixture to the time of ignition [3]. The prediction of this parameter is crucial for knock prevention and combustion phasing control. Although experimental bench testing can provide this information, it remains costly and time-consuming. Therefore, the main challenge is to design a numerical approach that is both accurate and computationally efficient, capable of reproducing the complex chemical and thermodynamic processes leading to auto-ignition.

In this context, the task of creating and validating such a model was entrusted to our research team, and this mission was assigned to me as part of this internship.

To achieve this, the internship focuses on:

- **Developing a numerical model** capable of predicting ignition delay time (IDT) for a given mixture under specified pressure and temperature conditions.
- **Validating the model** against available experimental data to ensure its accuracy and relevance.
- **Exploring the influence of operating parameters** (pressure, temperature, composition) on auto-ignition behaviour, using simulations to cover a wide range of conditions beyond what is feasible experimentally.

The goal is to provide a predictive tool that reduces reliance on physical testing while maintaining high fidelity in representing real engine phenomena. This tool will support faster fuel development cycles and contribute to the design of cleaner and more efficient spark-ignition engines.

3 Executed missions

3.1 Simulation

Pressure P [bar]	Temperature T [K]	Fuel/Air ratio Fr [-]	Mass fraction of O_2 Y_{O_2} [-]
$P_{\min} = 1$	$T_{\min} = 500$	$Fr_{\min} = 0.7$	$Y_{O_2, \min} = 0.01$
$P_{\max} = 100$	$T_{\max} = 1200$	$Fr_{\max} = 1.3$	$Y_{O_2, \max} = 0.23$

Table 2: Operating range of simulation parameters

The auto-ignition obviously depends on the physical parameters of the mixture and the combustion chamber. Among all these parameters, four were selected, as shown in Table 2. These conditions arise from both physical and empirical limitations. For instance, the air in the intake system cannot be richer in O_2 than it is in the atmosphere, therefore the mass fraction of O_2 cannot exceed 0.23. Similarly, the pressure cannot be lower than atmospheric pressure ($P_{atm} = 1.013$).

3.1.1 Simulation Tools - Cantera

Cantera is an open-source tool for chemical kinetics and thermodynamics. It provides a Python interface that makes it flexible [4]. In this internship project, I used Cantera 3.1.0 through its Python API to perform physical and chemical simulations.



Figure 4: Cantera

Below are the tasks handled by Cantera :

- Reaction Kinetics Simulation: I modelled the evolution of species concentrations over time.
- Thermodynamics Analysis: Computed properties such as temperature and pressure during reactions.
- Dynamic Simulations: Implemented time-dependent simulations with variable time steps to capture ignitions accurately
- Integration with Python: Leveraged Python's flexibility to automate simulations, visualize results, and analyze data.

VEMOD is related to both gasoline and diesel, but as those fuels are complex mixtures, modelling their behaviour in Cantera would require a very detailed chemical kinetic mechanism, which would make the simulation computationally very costly. For this reason, iso-octane (C_8H_{18}) was used as a surrogate fuel for gasoline and n-heptane (C_7H_{16}) as a surrogate for diesel. It is a well-established reference fuel that allows computationally efficient simulations [5]. Although iso-octane and n-heptane do not represent the exact composition of the fuels, they remain relevant for the understanding of the auto-ignition behaviour. Because the methodology is identical for n-heptane, only the results for iso-octane will be presented. The few differences in the results will be detailed

3.1.2 Method for detecting auto-ignition

To accurately determine the auto-ignition time of a fuel/air mixture, several approaches were investigated. Each method is based on different physical or chemical indicators of ignition, and an analysis of their advantages and limitations has been carried out. The objective was to identify the most reliable criterion under varying operating conditions.

Temperature Variation Method

At first glance, studying variations in the temperature of the mixture seemed to me to be the most intuitive way to determine whether the mixture will ignite. Indeed, the explosion of a fuel/air mixture is an exothermic reaction, which therefore releases energy in the form of heat.

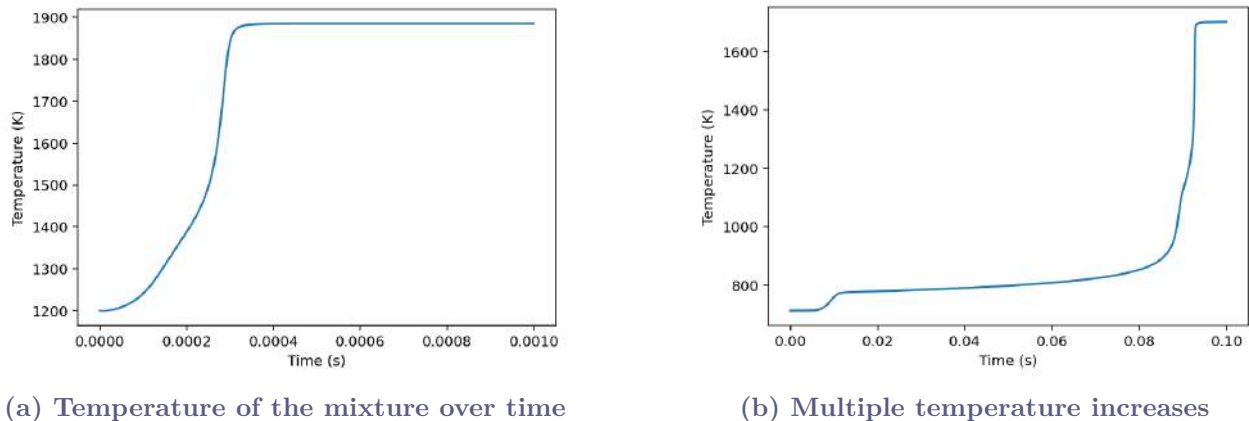


Figure 5: Different temperature variations as a function of time

The sudden increase in temperature in Figure 5a reflects the ignition of the mixture. Conditions are such that the mixture becomes kinetically unstable. By comparing the value of the derivative of the temperature with a well-defined threshold at any given time, it is possible to determine whether the mixture will ignite.

To obtain an accurate value for the auto-ignition time, this method remains limited as some cases show smaller explosions than others, this would therefore require adapting the threshold to each case.

There is also another problem that can skew the results of this method. In some cases as in Figure 5b, small temperature increases are observed at the very beginning of the reaction. If the threshold is too low, it may detect these increases as ignition events, when this is not the case.

Half-Rise Temperature Method

To obtain an accurate value for the auto-ignition time, the literature describes an empirical method that also focuses on the evolution of the temperature of the mixture. Using this method, the auto-ignition time is defined as the time at which the temperature increase has exceeded 50% of the total increase. This method provides equivalent precision regardless of the initial conditions of the mixture. However, this method seems sensitive to slow increases in temperature before the explosion, like in the previous method. These temperature increases, which do not indicate ignition, distort the IDT value because the ignition does not start at initial temperature but at a higher ‘floor’ temperature, which must therefore be determined.

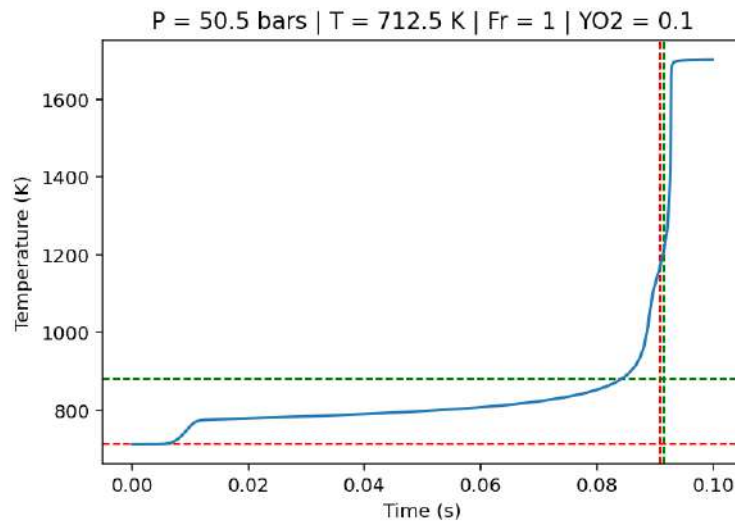


Figure 6: Difference between initial temperature and floor temperature

As highlighted with the dotted horizontal lines in Figure 6, the difference between the initial temperature and a floor temperature can be several tens of degrees. Not taking it into account would therefore significantly distort the result. In this case, the gap between the vertical lines representing the self-ignition time for this method is approximately 1 millisecond.

Hydroxyl Rate Method

Finally, the last method to consider is the hydroxyl (OH) rate of the mixture. In an air/fuel reaction, OH is an intermediate species that appears in the mixture during ignition. This is a good indicator of the IDT and it provides a more chemical approach to the problem [6] [7].

This criterion does not consider the heating of the mixture before ignition. Figure 7 shows that the OH peak occurs at the peak temperature, not at the start of the explosion. However, it is a very good way of checking the relevance and accuracy of the explosion peak, as it adds a chemical dimension to the previous physical criteria.

3.1.3 Optimal method and refinement

It remains risky to consider only one chemical or physical criterion, given the variety of cases that must be taken into account. The most robust method must therefore combine several of these criteria,

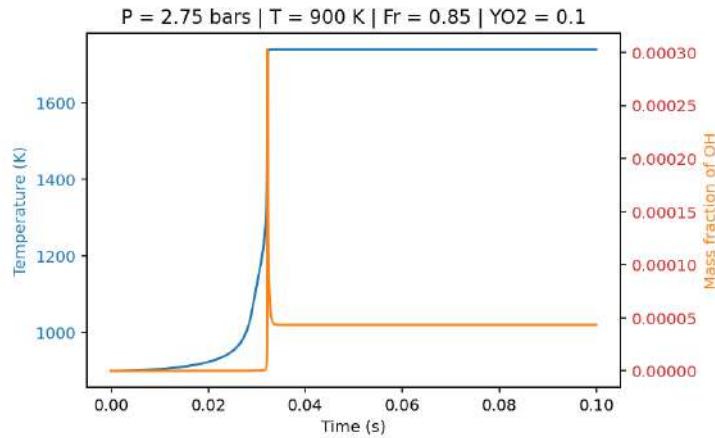


Figure 7: OH rate and temperature evolution over time

with a particularly precise time-step. Indeed, since highly accurate simulations are time-consuming, it was necessary to impose an adaptive time step. For a case where ignition occurs within a few microseconds, very high accuracy is required, otherwise the relative error with the true value of the auto-ignition time will be large. However, to maintain accuracy in cases where ignition occurs later and the time step has increased in the meantime, I had to implement a refinement method.

- First, the simulation is run to completion, with a very small time-step at the beginning to detect cases that explode very quickly, then increasingly larger as the simulation progresses.
- The Temperature Variation Method is used with a relatively high single value. At this point, the goal is not to obtain a precise value but to sort out cases where ignition actually occurs.
- Then, a first approximation of the IDT is obtained using the Half-Rise Temperature Method. Once again, at this stage we are not looking for the exact value, but only to get an idea of when the explosion is occurring. We do not yet have a floor temperature to obtain precise value for auto-ignition.
- From this initial approximation, we can look back at the beginning of the explosion and simulate this part again with a very precise time-step this time. This is the core of the refining method
- Finally, we can define a precise floor temperature in order to use the Half-Rise Temperature Method. We therefore obtain the IDT.

3.1.4 Dataset

At first, I did not know which places would later require more data, so given that the calculation time of the simulation was short enough thanks to the adaptive time-step, I simulated 10,000 points distributed evenly across the data range. We therefore had 10 values for each parameter. This choice is obviously not ideal but will be refined later. Indeed, as we will see in section [Accuracy of interpolators](#), this configuration was later replaced by a more suitable grid denser in the slow ignition zone which appears to be more chaotic.

3.2 Interpolation

Before we can obtain a self-ignition value that takes into account changes in physical and chemical parameters during an engine cycle, we must convert our discrete basis into a continuous grid. In fact, parameters such as gas pressure and temperature obviously do not change in steps, but continuously. Interpolating all of our data allows us to obtain an approximate function that gives us an estimated value of the IDT for any initial parameters.

Three different interpolation methods were tested:

- Radial Basis Function (RBF)
- Gaussian Process Regression (GPR)
- Neural Network (NN)

The aim was to test methods with different underlying mathematical concepts and compare them to retain only the most accurate ones. For each interpolator, accuracy was verified as follows. I chose random points in the area and simulate that point to compare the real value with the estimated value of the interpolator. Since I had to simulate the points I was comparing, the checks were limited. Therefore, the parameters of the interpolators had to be chosen meticulously.

Besides, interpolation only makes sense for cases where the mixture self-ignites, so I only used interpolators for this range of data.

The most representative measure to evaluate is the MRE (Mean Relative Error). Indeed, self-ignition can occur very quickly or very slowly. It can take anywhere from a microsecond to a tenth of a second. For that reason, we do not look at the MAE (Mean Absolute Error), but at the relative differences between predicted and actual values.

3.2.1 Radial Basis Function Interpolation

Radial Basis Function interpolation is a very powerful method for constructing multidimensional surfaces from scattered data points and does not require the data to be arranged on a regular mesh [8]. This method allows us to address our 4D-problem and permits an irregular database, which will later be the case. As detailed in the [Appendix](#), RBF Interpolator is a deterministic method, which means no statistical randomness is involved [9].

To optimize this interpolator, it is possible to adjust several main parameters:

- **Kernel:** Determines the shape of the radial basis function. It can be linear, cubic or even multiquadric, which influences the smoothness and interpolation behaviour.
- **Neighbors:** Limits interpolation to the nearest points instead of using all points.
- **Smoothing:** Introduces regularization to make the function less sensitive to noise.

One of its main advantages is that it produces continuous and smooth surfaces without requiring a predefined grid, making it particularly suitable for scattered data in multiple dimensions. Below is a figure illustrating the concept of RBF interpolation.

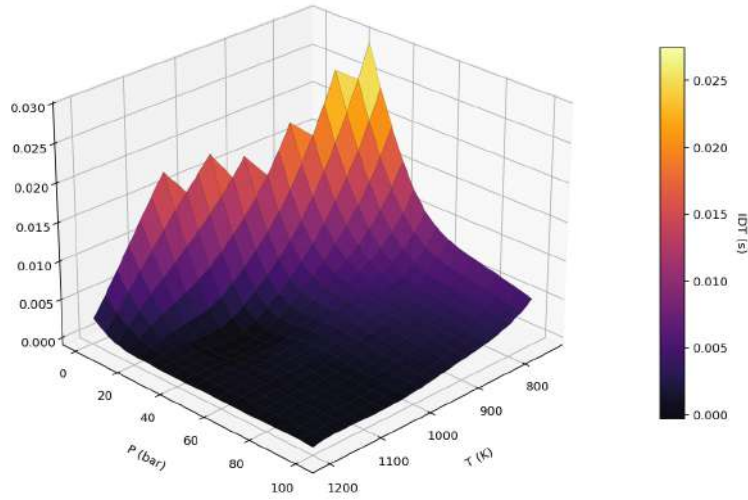


Figure 8: 2D RBF Interpolation (with fixed values of Fr and Y_{O_2})

3.2.2 Gaussian Process Regression Interpolation

Gaussian Process Regression is a probabilistic approach to regression used for modeling nonlinear relationships in data. Unlike RBF methods that assume a fixed functional form, GPR assumes that the function we want to predict behaves like a Gaussian process, which means that if we take any set of points, their values are related in a way that follows a Gaussian distribution [10].

Gaussian Process Regression (GPR) provides not only a predicted value at each point but also an associated confidence interval. This means that the model indicates how certain it is about each prediction, offering both an estimate and a measure of uncertainty, as illustrated in Figure 9

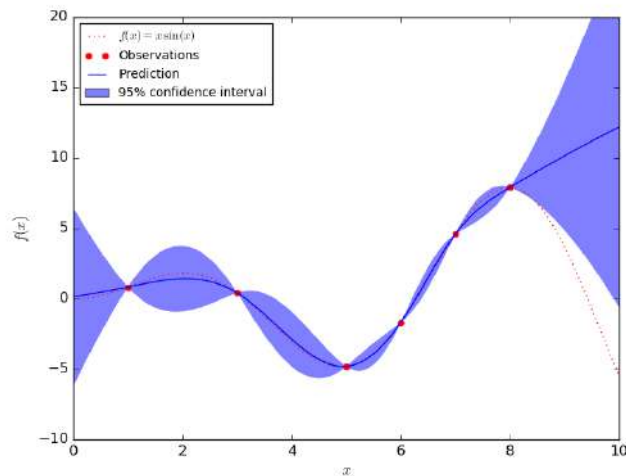


Figure 9: 1D GPR Interpolation

The main parameters that have been optimized are as follows:

- **Kernel:** Defines the covariance between any two data points [20]. It basically represents the similarity between two points. Kernels can be added and multiplied, which gives us many possible configurations to test.
- **Optimizer and Restarts:** The number of restarts allows the optimizer to find the best set of hyper-parameters despite the bad initial values.

3.2.3 Neural Network

A Neural Network is a machine learning model composed of layers of interconnected units, called neurons, whose connections are associated with weights. Neurons operate by performing basic computation, such as weight linear transformation of the inputs and nonlinear activation function [11]. The training process adjusts these weights to minimize the prediction error.

The key optimized parameters are as follows:

- **Learning rate (lr):** During training, the step size for weight updates is determined by the learning rate. A lower value ensures stable convergence, while a higher value speeds up learning but risks instability.
- **Weight Decay:** A regularization term that penalizes large weights to prevent overfitting and improve generalization.
- **Number of Epochs:** Defines how many times the entire dataset is passed through the network during training.
- **Hidden Layer Sizes:** The number of neurons in each hidden layer controls the model's capacity to learn complex patterns.
- **Batch Size:** The number of samples processed together before updating weights; affects training speed and stability.
- **K-Folds:** Refers to cross-validation splits used to evaluate model performance and robustness.

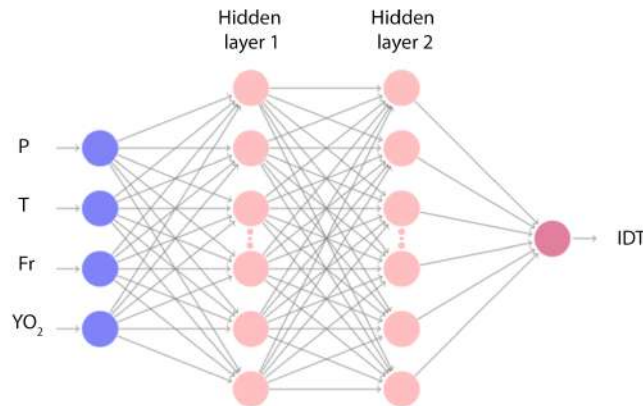


Figure 10: OH rate and temperature evolution over time

The optimized neural network architecture is composed by two hidden layers with a large number of neurons: 165 neurons for the first layer and 80 neurons in the second. Figure 8 illustrates the structure of this neural network.

This configuration hasn't been chosen arbitrarily, it has been determined through an optimization algorithm [12]. I conducted a series of 50 successive trials, each exploring variations around the previously most accurate configuration. Among the tested architectures were for instance neural networks with three hidden layers and others with significantly fewer neurons in the layers.

3.3 Results

3.3.1 Accuracy of interpolators

The results obtained using the various interpolation methods highlighted the behaviour of the mixture. In fact, two areas can be visually distinguished in which the behaviour of the mixture is completely different. This can be seen in Figure 11.

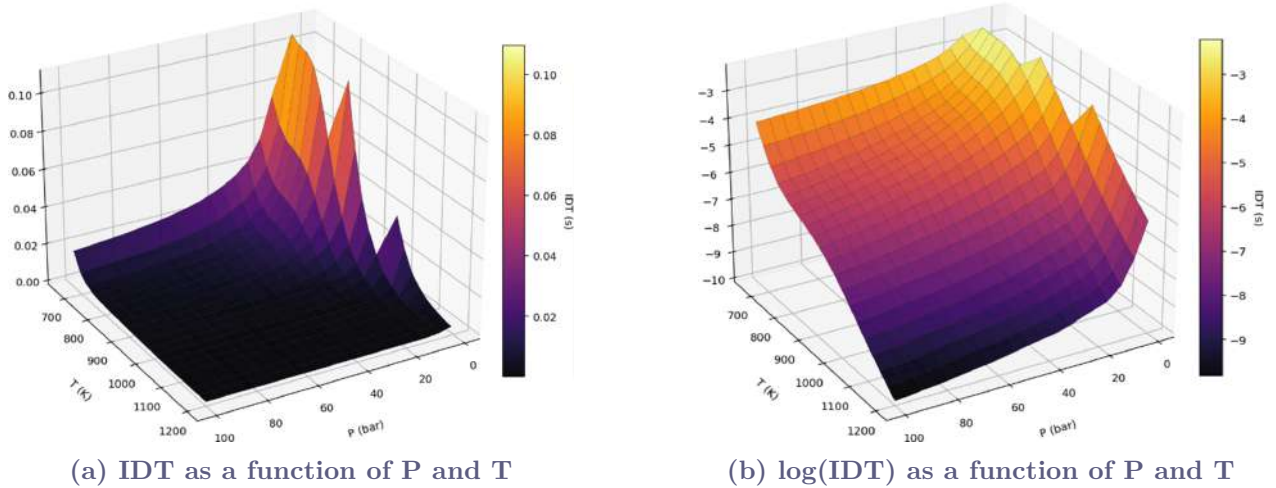


Figure 11: IDT and $\log(\text{IDT})$ as a function of P and T

The graph on the left shows the IDT of the mixture depending on its initial pressure and temperature (Fr and Y_{O_2} were set arbitrarily, but the behaviour stays the same regardless of their value). There is a very steep slope for points with pressure below 20/25 bar, and a flat section for high values of pressure and temperature. To grasp the differences between two such small values, I felt it necessary to use the logarithm function on the IDT. The results using the logarithm function are shown on the graph on the right. It already seems much more effective to interpolate such data.

Thanks to the RBF interpolator, which is the quickest to set up, I was able to observe its accuracy depending on the interpolated area, in order to provide more data where necessary (as announced in section Dataset).

The results shown in Figure 12 are consistent with those in Figure 11. On a first hand, despite the

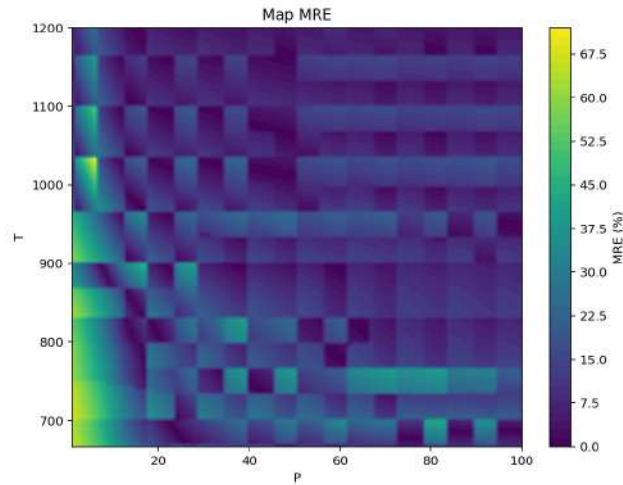


Figure 12: MRE mapping based on pressure and temperature

logarithm, we observe that the points located on the slope in Figure 11 appear to be more difficult to interpolate, as their relative error can be very high (up to ~60%). On the other hand, the points located on the flat area have a lower relative error. These results enabled a densification of the database by adding points (2,500) for which the interpolator is least accurate.

Given the hybrid behaviour of the model, I decided to treat two distinct areas separately.

- Zone 1: points for which $P < 20$ bar
- Zone 2: points for which $P \geq 20$ bar

Tables 4 and 3 below show the results:

MRE (%)	Zone 1	Zone 2
RBF Interpolator	11	1.3
GPR Interpolator	1.97	3.52
Neural Network	5.39	5.92

Table 3: MRE for each zone

Species	MRE (%)	MAE (ms)
Isotane - C ₈ H ₁₈	1.82	0.12
N-heptane - C ₇ H ₁₈	1.23	0.05

Table 4: MSE — Table 2

First, the results in 3 (which only concerns the isooctane) show that NN is not very accurate, regardless of the area. This can be explained by the lack of data available to the neural network [13]. This is because it is a 4-dimensional problem, which requires a lot of data. Having 12,000 points is sufficient to obtain a fairly accurate result, but not enough to be more accurate than RBF. Next, we observe that GPR is most accurate in zone 1. This zone is the most complex because it has a steep slope and the area is not necessarily covered in the same way everywhere (since densification has been carried out), which is why GPR, which is probabilistic, is more accurate. Conversely, RBF is very effective on Zone 2, which is much flatter, smoother, and without noise. As mentioned above, I was also asked to obtain a model using n-heptane instead of isooctane. The method was exactly the

same, but the slightly different behaviour of the n-heptane made be only use the GPR to interpolate (the most accurate in both zones). Table 4 shows the MRE and MAE for each species.

The models are clearly very accurate and provide a good estimate of the auto-ignition time of a mixture. Having such a low MAE and MRE not only reflect the validity of the method I used, but above all the effectiveness of the chosen interpolations.

3.3.2 IDT for variable parameters

For a mixture at constant pressure, the auto-ignition time is now known fairly precisely:

$$\tau(T, P, Fr, Y_{O_2})$$

But realistically, the pressure also increases as the reaction progresses. To obtain the auto-ignition time value with variable parameters, I was therefore asked to implement the Cumulative Ignition Delay (CID) method, otherwise called the Inverse Livengood-Wu approach [14]:

$$\int_0^{t_{ign}} \frac{1}{\tau(T(t), P(t), Fr, Y_{O_2})} dt = 1$$

This method consists of calculating the cumulative probability of ignition. By comparing this value to 1, we obtain a time for which we consider that ignition has occurred with a probability of approximately 63% [14].

Now, for any initial parameters, we have a value of the IDT of the mixture that takes into account the evolution of these parameters during the simulation.

3.3.3 Verification on a real engine cycle

For technical reasons, I couldn't test the accuracy of my results on the CMT test benches. However, a numerical solution was proposed to me. Indeed, using Cantera, I was able to simulate a real engine cycle. Only certain phenomena, such as heat transfers to the walls or turbulences, were not modelled. Below are the parameters for this engine (chosen arbitrarily).

Measure	Symbol	Unit	Value
Displacement	V_d	<i>cc</i>	500
Bore	D	<i>mm</i>	86
Stroke	s	<i>mm</i>	86
Compression ratio	CR	—	18
Engine speed	Ω	<i>rpm</i>	2000

Table 5: Engine parameters

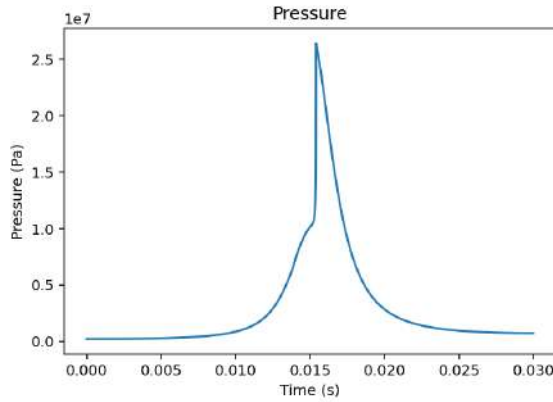
I chose to start the simulation at the BDC so that before compression, the mixture is homogeneous and the reaction has not begun yet. The initial parameters are much lower this time, because while the simulations used for interpolation were meant to cover a wide range of conditions and provide

IDT values throughout the entire thermodynamic path of the mixture, this new simulation requires realistic initial conditions corresponding to BDC.

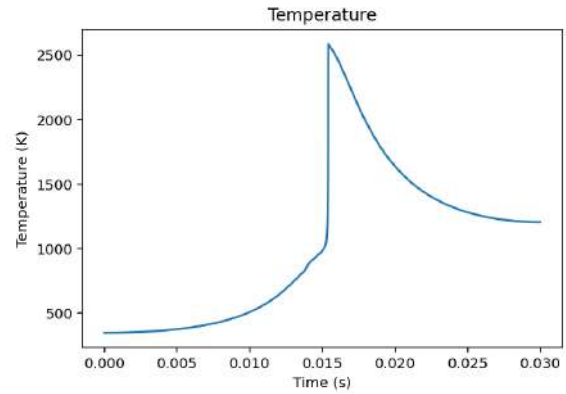
Pressure P [bar]	Temperature T [K]	Fuel/Air ratio Fr [-]	Mass fraction of O_2 Y_{O_2} [-]
$P_{\min} = 1$	$T_{\min} = 300$	$Fr_{\min} = 0.7$	$Y_{O_2, \min} = 0.01$
$P_{\max} = 3$	$T_{\max} = 500$	$Fr_{\max} = 1.3$	$Y_{O_2, \max} = 0.23$

Table 6: BDC parameters range

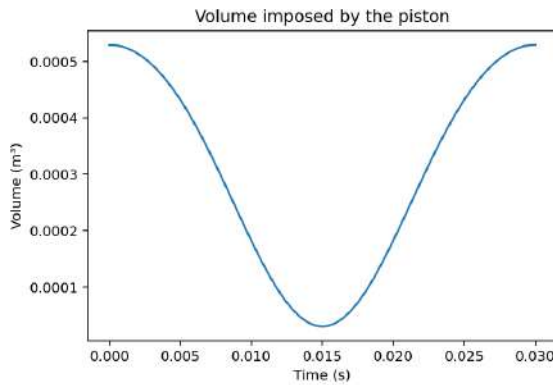
This allowed me to compare the differences between the IDT predicted using CID method and the actual IDT obtained by analysing the evolution of the parameters at numerous points. Presented herein are graphs depicting the evolution of parameters over an engine cycle.



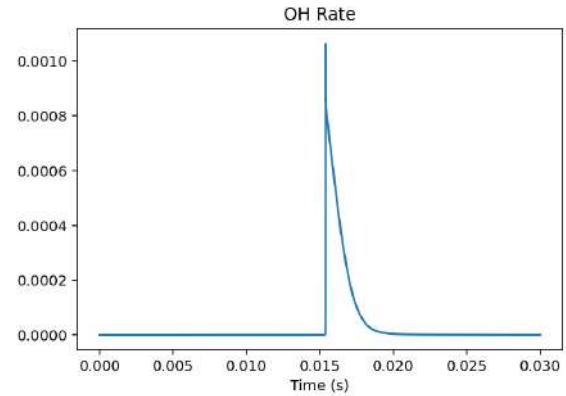
(a) Pressure



(b) Temperature



(c) Volume



(d) OH rate

Figure 13: Evolution of pressure, temperature volume and OH rate over an engine cycle

Figure 13 shows (with the following initial parameters: $P = 2\text{bar}$, $T = 350\text{K}$, $Fr = 1$, $Y_{O_2} = 0.15$) the auto-ignition of the mixture. The temperature rises suddenly before the end of piston compression. At this precise moment, OH appears in the mixture. These graphs show what is considered to be the IDT, as we look precisely at the explosion in temperature and OH rate.

3.4 Post-processing of bench tests results

In parallel with the main project I led, presented above, several tests were carried out to test some aspects of the HVO fuel. In this case, Mr. Daniel Robayo Rueda, who supervised me most of the time, was conducting research on particulate emission at the engine exhaust. We therefore carried out a series of tests, which I was able to post-process on MATLAB.

Among the different tests carried out during my work, I was mainly involved as an assistant. However, I had the opportunity to do the post-processing of one specific test, which consisted of particle emission measurements on a diesel engine. The objective of this study was to compare engine performance when operating with HVO. Below are the graphs illustrating the results of the post-processing :

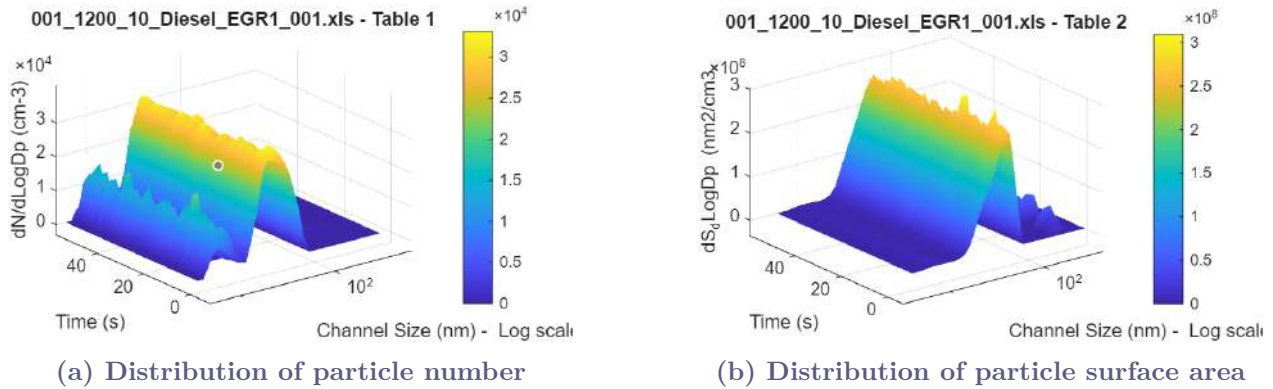


Figure 14: Time-resolved particle number size distribution

During the internship, particle emissions were measured using conventional diesel fuel at several representative operating points to establish a baseline. The same measurements with HVO under identical conditions are planned to be carried out after the internship, enabling a direct comparison between the two fuels. The post-processing work I carried out concerned only the first phase of the test campaign, namely the measurements conducted with diesel fuel.

As shown in Figure 14, the objective was to represent the particle size distribution ($dN/d\text{Log}Dp$) over a given period of time. The test bench is equipped with a particle analyzer, which measures both the quantity of emitted particles and their size distribution. This figure shows one of the nine operating points investigated, corresponding to engine operation at 1200 rpm and 10% load with the Exhaust Gas Recirculation (EGR) system operational.

Once the tests with HVO are completed, the MATLAB routine developed during this internship will

be used to generate the same set of graphs as for diesel. This will allow a direct comparison between the two fuels and provide deeper insights into the behaviour of HVO under identical operating conditions.



Figure 15: Engine test bench

4 Conclusion

The main goal of my internship was to develop a numerical model for the prediction of auto-ignition in spark ignition engines. This work contributes to ongoing research on alternative fuels, mainly on HVO. It involved a detailed chemical simulation using Cantera, as well as the creation of a database and the use of various interpolation methods. This led to a tool that can predict with good accuracy ignition delay times over a wide range of operating conditions and standard engine cycles.

Although the results are promising, we cannot fully rely on the model until it is validated with a real engine. Without experimental verification on a test bench, we must interpret the conclusions of the numerical work with caution. It is essential to compare simulated predictions with actual engine measurements to evaluate the accuracy of the model. These tests should be carried out in 2026, if there are no more technical issues.

Furthermore, the behaviour of the fuel we studied is complex and not completely understood. Combustion and auto-ignition involve complicated chemical and thermodynamic processes. There is still room for further investigation and improvement. I hope this work will become a strong contribution for the future of this research project.

From a professional point of view, this internship has been a very positive experience. It allowed me to take on responsibilities of a research task from its definition to its execution. I explored different methods and learned a lot from my mistakes. I developed my ability to question myself, think critically about results, and adjust strategies when needed. Working in a research team also improved my communication skills. Indeed, I improved my technical English, especially in speaking. I've become more comfortable with programming in Python, *C++* and *MATLAB*. Finally, it deepened my understanding of internal combustion engines, alternative fuels, and more generally of the automotive field.

Last but not least, my main motivation for this internship was to know whether or not the research could appeal to me, while working on a topic related to my career plan. Even though the project itself fully met my expectations, I realized that I wouldn't be able to make my career in research. I feel the need to know precisely where I am going, to deal with competition and confront my work with others' work. This internship strengthened my belief in working in the motor-sport industry, to get this extra motivation from competition.

Nomenclature

Abbreviations

BDC	Bottom Dead Center
C7H16	N-heptane
C8H18	Iso-octane
CFD	Computational Fluid Dynamics
CI	Compression ignition
CID	Cumulative ignition delay
CMT	Clean Mobility & Thermofluids
CPR	Constant pressure reactor
EGR	Exhaust Gas Recirculation
Fr	Fuel/air ratio
GPR	Gaussian Process Regression
H2	Dihydrogen
HVO	Hydrotreated Vegetable Oil
ICE	Internal combustion engine
IDT	Ignition delay time
lr	Learning rate
MAE	Mean average error
MRE	Mean relative error
NN	Neural Network
O2	Dioxygen
OH	Hydroxyl
RBF	Radial Basis Function
SI	Spark Ignition
TDC	Top Dead Center
UPV	Universitat Politècnica de València
VEMOD	Virtual Engine Model

Symbols

$\dot{m}$	Mass flow rate	$[kg.s^{-1}]$
ϵ	Collision efficiency	$[-]$
ϕ	Radical function	$[-]$
A	Pre-exponential factor	$[Pa]$
b	Temperature exponent	$[-]$
C	Concentration	$[mol.L^{-1}]$
E_a	Activation energy	$[J.mol^{-1}]$
H	Enthalpy	$[J]$
k_f	Forward rate constant	$[mol^{1-r_1-r_2}.s^{-1}.cm^{-3}]$
m	Mass	$[kg]$
P	Pressure	$[Pa]$
R	Universal gas constant	$[J.mol^{-1}.K^{-1}]$
R_f	Forward reaction rate	$[mol.s^{-1}.cm^{-3}]$
T	Temperature	$[K]$
Y_k	Mass fraction	$[-]$
Y_{O_2}	Mass fraction of O_2	$[-]$

List of Figures

1	CMT's infrastructure	5
2	Distribution of CMT's funds	6
3	4-Stroke SI engine cycle	8
4	Cantera	11
5	Different temperature variations as a function of time	12
6	Difference between initial temperature and floor temperature	13
7	OH rate and temperature evolution over time	14
8	2D RBF Interpolation (with fixed values of Fr and Y_{O_2})	16
9	1D GPR Interpolation	16
10	OH rate and temperature evolution over time	17
11	IDT and $\log(IDT)$ as a function of P and T	18
12	MRE mapping based on pressure and temperature	19
13	Evolution of pressure, temperature volume and OH rate over an engine cycle	21
14	Time-resolved particle number size distribution	22
15	Engine test bench	23

List of Tables

1	Status of research projects and doctoral theses	6
2	Operating range of simulation parameters	11
3	MRE for each zone	19
4	MSE — Table 2	19
5	Engine parameters	20
6	BDC parameters range	21
7	State variables for CPR	29
8	Most classical Kernels	30
9	Neural Network parameters	31

References

- [1] Université Mila. *Overview of Internal Combustion Engines: Components, Classification, and Operating Principles*. URL: https://elearning.centre-univ-mila.dz/a2024/pluginfile.php/25824/mod_resource/content/0/Chapitre%20I.pdf. (accessed: 20.01.2026).
- [2] Vicente Macián et al. “Characterization of the turbulent flame front surface in spark ignition engines during spark ignition operation to identify controlled auto-ignition and abnormal combustion”. In: *International Journal of Engine Research* 22 (2020), pp. 2149–2168. DOI: 10.1177/1468087420933780.
- [3] Boyang Su et al. “Dependence of ignition delay time on its definition - A case study on methane ignition”. In: *ScienceDirect Combustion and Flame* 262 (2024). DOI: 10.1016/j.combustflame.2024.113364.
- [4] Cantera Developers. *Cantera: Complete Documentation, API, Installation Guide and Tutorials*. URL: <https://cantera.org/>. (accessed: 20.01.2026).
- [5] Marco Mehl et al. “Kinetic modeling of gasoline surrogate components and mixtures under engine conditions”. In: *Proceedings of the Combustion Institute* 33 (2011), pp. 193–200. DOI: 10.1016/j.proci.2010.05.027.
- [6] Baptiste Sirjean et al. “The autoignition of cyclopentane and cyclohexane in a shock tube”. In: *PROCEEDINGS OF THE COMBUSTION INSTITUTE* 31 (2007), pp. 277–284. DOI: 10.48550/arXiv.0706.2060.
- [7] Cantera Developers. *Shock Tube Ignition Delay Example: Tracking OH Radical to Determine Ignition Delay*. URL: https://cantera.org/dev/examples/python/reactors/non_ideal_shock_tube.html. (accessed: 20.01.2026).
- [8] DataMListic. *RBF Kernel Explained: Mapping Data to Infinite Dimensions*. URL: https://youtu.be/tV5X77lGYP4?si=oZ2_-j92cewSPkHI.
- [9] Wikipedia contributors. *Radial basis function interpolation*. 2025.
- [10] J. Wang. *An intuitive tutorial to Gaussian process regression*. <https://www.biostat.wisc.edu/bmi776/papers/GPR2.pdf>. 2020.
- [11] 3Blue1Brown. *But what is a neural network? / Deep learning chapter 1*. URL: <https://www.youtube.com/watch?v=aircAruvnKk>. (accessed: 20.01.2026).
- [12] Optuna Contributors. *Optuna Documentation*. Accessed: 2026-01-21.
- [13] Th.I Götz et al. “Number of necessary training examples for Neural Networks with different number of trainable parameters”. In: *Proceedings of the Combustion Institute* 13 (2022). DOI: 10.1016/j.proci.2010.05.027.
- [14] P. Livengood and W. Wu. “Livengood-Wu Integral Correlation: Predicting Autoignition Timing via Integration of Local Delays”. In: *Combustion and Flame* 9 (1955), pp. 1–15. DOI: [https://doi.org/10.1016/0010-2180\(55\)90025-4](https://doi.org/10.1016/0010-2180(55)90025-4).
- [15] 3Blue1Brown. *Gradient descent, how neural networks learn / Deep learning chapter 2*. URL: https://youtu.be/IHZwWFHWa-w?si=9lpw_7o8KeHTLgTC. (accessed: 22.01.2026).

5 Appendix

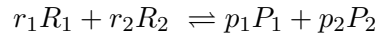
Physical and chemical basis of the Cantera simulation method

Cantera provides a comprehensive framework for handling detailed chemical kinetics. In our case, the combustion of hydrocarbon fuel such as isooctane involves a large number of elementary reactions and intermediate species. Cantera evaluates the rate of each reaction in the mechanism and each time step, whose combination governs the temporal evolution of the chemical composition of the mixture.

- **Elementary Reactions**

Most reactions in the mechanism are treated as elementary gas-phase reactions. Their forward reaction rate follows the law of mass action.

For a general reaction:



The forward reaction rate R_f is calculated as

$$R_f = k_f [R_1]^{r_1} [R_2]^{r_2}$$

where k_f is the forward rate constant and is determined from the modified Arrhenius form:

$$k_f = AT^b e^{-E_a/RT}$$

Here, A is the pre-exponential factor, T is the temperature, b is the temperature exponent, E_a is the activation energy, and R is the universal gas constant.

- **Three-body reactions**

Although most reactions are treated as elementary reactions, three-body reactions can occur during combustion. In this case, the forward reaction rate R_f is calculated as

$$R_f = [R_1][R_2][M]k_f(T) \quad \text{with} \quad [M] = \sum_k \epsilon_k C_k$$

where M represents an unspecified collision partner that carries away excess energy to stabilize the molecule $R_1 R_2$ (forward direction) or supplies energy to break the $R_1 R_2$ bond (reverse direction), and ϵ and C the collision efficiency and the concentration of each species respectively.

To simulate a chemical reaction, we need a reactor. It defines the environment in which the reaction takes place. Several reactor types are available, such as Ideal Gas Reactor, Constant Pressure Reactor (CPR), or Control Volume Reactor. In our case, we must use the Constant Pressure Reactor, as it helps to study the intrinsic chemical behaviour of the mixture, without being influenced by the mechanical effects such as compression.

- **Constant Pressure Reactor**

A constant pressure reactor is defined by the state variables below

Measure	Symbol	Unit
Mass of the reactor's contents	m	kg
Total enthalpy of the reactor's contents	H	J
Mass fractions for each species	Y_k	-

Table 7: State variables for CPR

The CPR is governed by 3 equations. The mass conservation equation, the species equations and the energy equation.

- First, The total mass of the reactor's contents changes as a result of flow through the reactor's inlets and outlets, and production of homogeneous phase species on surfaces:

$$\frac{dm}{dt} = \sum_{in} \dot{m}_{in} - \sum_{out} \dot{m}_{out} + \dot{m}_{wall}$$

- Then, Cantera computes the rate of change in the mass of each species:

$$m \frac{dY_k}{dt} = \sum_{in} \dot{m}_{in} (Y_{k,in} - Y_k) + \dot{m}_{k,gen} - Y_k \dot{m}_{wall}$$

where $\dot{m}_{k,gen}$ represents the rate at which the species is generated.

- Finally, the First Law of Thermodynamics for an open system gives us:

$$\frac{dU}{dt} = -p \frac{dV}{dt} + \dot{Q} + \sum_{in} \dot{m}_{in} h_{in} - h \sum_{out} \dot{m}_{out}$$

By differentiating the definition of the enthalpy $H = U + PV$ and noting that in our case the pressure remains constant so $\frac{dp}{dt} = 0$, the energy equation becomes:

$$\frac{dH}{dt} = \dot{Q} + \sum_{in} \dot{m}_{in} h_{in} - h \sum_{out} \dot{m}_{out}$$

RBF Interpolator

A Radial Basis Function interpolator is used to find a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ that interpolates the data:

$$f(x_i) = y_i \quad \text{for all } i,$$

and whose value at a point x depends on the data points only through the distance $\|x - x_i\|$.

This method assumes that the interpolating function can be written as follow:

$$f(x) = \sum_i \lambda_i \phi(\|x - x_i\|)$$

where ϕ is the radical function (called Kernel in our case), and λ_i are coefficients determined by enforcing the interpolation conditions. Therefore, to get a precise interpolation, we need to get both an appropriate radical function and as much data as possible to determine the λ_i . Below are the most widely used Kernels:

Kernel	Formula
Gaussian	$e^{-(\epsilon r)^2}$
Multiquadric	$\sqrt{1 + (\epsilon r)^2}$
Inverse-Multiquadric	$1/\sqrt{1 + (\epsilon r)^2}$
Cubic	r^3

Table 8: Most classical Kernels

The interpolation has been tested with these 4 radical functions and ended up choosing the cubic one as it is the best in our case. Other parameters have been also tested, such as the number of neighbors considered in the radical function or a smoothing parameter to avoid overfitting.

GPR Interpolator

Gaussian Process Regression is a regression method used to infer an unknown function from data, while quantifying uncertainty. Instead of considering a parametric form for the function $f(x)$, GPR assumes that it is a random function:

$$f(x) \sim \mathcal{GP}(\mathbb{E}[f(x)], k(x, x'))$$

where $\mathbb{E}[f(x)]$ is the mean function and $k(x, x')$ defines the covariance function (called Kernel in our case). The Kernel defines how correlated the values are. We therefore need to find the most appropriate Kernel. I tried several Kernels that I could sum and add in order to get the most

precise regression. Overall, the kernel is composed of a constant kernel, a white noise kernel, and two Matérn kernels.

Neural Network

A neural network is a model that learns from a set of data. It is made up of several layers of neurons that are connected to each other using weights. Each neuron performs a sum of its inputs, optionally passes it through a non-linear activation function σ , and outputs a value to the next layer. Mathematically, for a single hidden layer:

$$h = \sigma(W_1x + b_1), \quad \hat{y} = W_2h + b_2$$

where x is the input, b_1 and b_2 are biases, W_1 and W_2 are the weight matrices, and $\hat{y}$ is the output.

The Neural Network trains itself by changing the weights. To know which weights to change, and to what value, we use the gradient descent method [15]. It helps adjusting those weights to reduce a chosen loss function; the more this loss function is reduced, the more the neural network is accurate.

In our case, we want to get the lowest values of MRE and MAE. However, while lowering the MAE would create zones where the MRE is high (low IDT zones, because a high MRE can still be a low MAE in these zones), lowering the MRE would be the most efficient way to get a precise neural network.

At each step, we compute these values and the neural network changes its weights to obtain a lower value of this loss function for the point we just tested.

To help the network to learn efficiently, several parameters need to be adjusted:

Parameter	Value
Learning rate	?
Number of epochs	50
Batch size	?
First layer size	165
Second layer size	80
K-flods	3
Weight decay	?

Table 9: Neural Network parameters

These parameters are not chosen manually. We use an algorithmic approach to systematically explore different architectures and find the best one [12]. This method can be very powerful, but it requires a lot of points to accurately adjust the weights.

Codes

This first code is the simulation using Cantera. It contains the "autoignition_test" function, which implements the method described in [Optimal method and refinement](#)

```
1 import cantera as ct
2 import time
3 import openpyxl
4 import numpy as np
5 import math
6 import warnings
7 warnings.filterwarnings("ignore", category=UserWarning)
8
9 # Files
10
11 Fuel = 'IC8H18'
12 File = "Isoctane data/gasoline surrogate mech.yaml"
13 FileExport = "Main/"
14
15 # Parameters
16
17 tmax=1
18 t1 = np.arange(0, 100e-6 + 1e-6, 1e-6)
19 t2 = np.arange(100e-6 + 10e-6, 1e-3 + 1e-6, 10e-6)
20 t3 = np.arange(1e-3 + 0.1e-3, 0.1 + 1e-6, 0.1e-3)
21 t4 = np.arange(0.1+0.01, 1+0.01, 0.01)
22 t_total = np.concatenate([t1, t2, t3,t4])
23
24 MaxVariation = 10e5
25
26 Pmin = 1 # Bar
27 Pmax = 100 # Bar
28 Tmin = 600 # Kelvin
29 Tmax = 1200 # Kelvin
30 Frmin= 0.7
31 Frmax= 1.3
32 Y02min = 0.1
33 Y02max = 0.23
34
35 # Gets the O2 mole rate from the O2 mass rate
36
37 def Y02toX02(Y02):
38     M02=31.9988
39     MN2=28.0134
40     YN2 = 1 - Y02
41     N02 = Y02/M02
42     NN2 = YN2/MN2
43     return N02 / (N02 + NN2)
44
45 # Returns the IDT (Optimal method)
46
47 def autoignition_test(P,T, Fr, Y02, File, Fuel):
```

```

48
49 # Gas + Reactor
50
51 gas = ct.Solution(File)
52 X02 = Y02toX02(Y02)
53 oxidizer = {'O2' : X02, 'N2' : 1-X02}
54 gas.set_equivalence_ratio(Fr, Fuel, oxidizer)
55 gas.TP = T, P
56 reactor = ct.ConstPressureReactor(gas)
57 sim=ct.ReactorNet([reactor])
58
59 # Full Simulation to assess if the mixture ignites
60
61 try :
62     Temp = [T]
63     Autoignition = False
64     for i in range(1,len(t_total)):
65         sim.advance(t_total[i])
66         if (gas.T - Temp[-1])/(t_total[i] - t_total[i-1]) >
            MaxVariation :
67             Autoignition = True
68             Temp.append(gas.T)
69
70 except ct.CanteraError :
71     print("1st simulation : Error")
72     print(P,T,Fr,Y02)
73     return autoignition_test(P, T+0.001, Fr, Y02, File, Fuel)
74
75
76 # First Approximation of the IDT
77
78 IDT = tmax # By default
79
80 if Autoignition:
81     Tmax = np.max(Temp)
82     Tmin = np.min(Temp)
83     T50 = Tmin + 0.5 * (Tmax - Tmin)
84     i_T50 = np.argmax(Temp > T50)
85     dT = np.gradient(Temp, t_total)
86     Threshold = 1e4
87
88     i = i_T50
89     T_plateau = Temp[i]
90     while i > 0 and dT[i] > Threshold:
91         i -= 1
92
93     # Refined Simulation to get a more precise value of IDT
94     gas2 = ct.Solution(File)
95     gas2.set_equivalence_ratio(Fr, Fuel, oxidizer)
96     gas2.TP = T, P
97     reactor2 = ct.ConstPressureReactor(gas2)
98     sim2=ct.ReactorNet([reactor2])

```

```
99
100     t2=t_total[math.floor(0.95*i)]
101     TempAroundIgnition=[gas2.T]
102
103     try :
104         while gas2.T < T50:
105             t2+=1e-6
106             sim2.advance(t2)
107             TempAroundIgnition.append(gas2.T)
108
109             dT = np.gradient(TempAroundIgnition,1e-6)
110             Treshold2=1e3
111             i = len(dT)-1
112
113             while i>0 and dT[i]>Treshold2:
114                 i-=1
115
116             T_plateau = TempAroundIgnition[i]
117
118     except ct.CanteraError :
119         print("2nd simulation : Error")
120         print(P,T,Fr,YO2)
121
122     T50_refined = T_plateau + 0.5 * (Tmax - T_plateau)
123     i_IDT = np.argmax(Temp > T50_refined)
124     IDT = float(t_total[i_IDT])
125
126     return IDT
```

This second code is used for the creation of the GPR Interpolator as well as the RBF Interpolator. It creates at the end, an "interpolator" function, that separates the two distinct areas as mentioned in the [Accuracy of interpolators](#) section.

```
1     import numpy as np
2 import pandas as pd
3 from sklearn.gaussian_process import GaussianProcessRegressor
4 from sklearn.gaussian_process.kernels import WhiteKernel, ConstantKernel,
   Matern
5 from sklearn.preprocessing import StandardScaler
6 import time
7 from joblib import dump, load
8 from scipy.spatial import Delaunay
9 from sklearn.cluster import KMeans
10
11 File = 'IC8H18.xlsx'
12 FileFuel = "Isoctane data/gasoline surrogate mech.yaml"
13 Fuel = 'IC8H18'
14
15 def Init_GPR(File = File, Fuel = Fuel, subset_size=3000, test_size=500,
   retrain=False, use_kmeans=True):
16     Start = time.time()
17
18     # Data loading
19
20     cols = ['P', 'T', 'Fr', 'Y02']
21     df = pd.read_excel(File)
22     df = df[df['IDT'] < 1].copy()
23
24     X = df[cols].values
25     y = df['IDT'].values
26     y_log = np.log(y)
27
28     # Scaling
29
30     scaler = StandardScaler()
31     X_scaled = scaler.fit_transform(X)
32
33     # Subsets
34
35     if len(X_scaled) > subset_size:
36         if use_kmeans:
37             kmeans = KMeans(n_clusters=subset_size, random_state=31,
   n_init=10)
38             labels = kmeans.fit_predict(X_scaled)
39
40             closest_indices = []
41             for i in range(subset_size):
42                 cluster_points = np.where(labels == i)[0]
43                 if len(cluster_points) == 0:
44                     continue
45                 center = kmeans.cluster_centers_[i]
```

```

46         dists = np.linalg.norm(X_scaled[cluster_points] - center,
47                                 axis=1)
48         closest = cluster_points[np.argmin(dists)]
49         closest_indices.append(closest)
50
51     idx_train = np.array(closest_indices, dtype=int)
52 else:
53     idx_train = np.random.choice(len(X_scaled), subset_size,
54                                 replace=False)
55 else:
56     idx_train = np.arange(len(X_scaled), dtype=int)
57
58 all_idx = np.arange(len(X_scaled))
59 test_candidates = np.setdiff1d(all_idx, idx_train)
60 if len(test_candidates) < test_size:
61     test_size = len(test_candidates)
62 idx_test = np.random.choice(test_candidates, size=test_size, replace=
    False)
63
64 X_train, y_train_log = X_scaled[idx_train], y_log[idx_train]
65 X_test, y_test_log = X_scaled[idx_test], y_log[idx_test]
66
67 # Training
68
69 if retrain:
70     kernel = ConstantKernel(1.0) * (Matern(length_scale=np.ones(4), nu
71                                         =1.5) + Matern(length_scale=0.1*np.ones(4), nu=2.5)) +
72         WhiteKernel(1e-6)
73
74     gpr = GaussianProcessRegressor(kernel=kernel, n_restarts_optimizer
75                                   =2, normalize_y=True, random_state=0)
76
77     gpr.fit(X_train, y_train_log)
78
79     y_pred_log = gpr.predict(X_test)
80     y_pred = np.exp(y_pred_log)
81     y_true = np.exp(y_test_log)
82
83     mae = np.mean(np.abs(y_true - y_pred))
84     mre = np.mean(np.abs(y_true - y_pred) / y_true) * 100
85
86     print(f"MAE (test) = {mae:.6f} s")
87     print(f"MRE (test) = {mre:.2f} %")
88
89 # Saving
90
91     dump(gpr, 'gpr_model_' + str(Fuel) + '.joblib')
92     dump(scaler, 'scaler_' + str(Fuel) + '.joblib')
93     dump(idx_train, 'subset_indices_' + str(Fuel) + '.joblib')
94 else:

```

```
92     gpr = load('gpr_model_' + str(Fuel) + '.joblib')
93     scaler = load('scaler_' + str(Fuel) + '.joblib')
94     try:
95         idx_train = load('subset_indices_' + str(Fuel) + '.joblib')
96     except:
97         idx_train = np.arange(len(X_scaled), dtype=int)
98
99
100     param_array_scaled = scaler.transform(df[cols].values)
101     delaunay = Delaunay(param_array_scaled)
102
103     End = time.time()
104     print(f"Total time = {End - Start:.2f} s")
105
106     print(gpr.kernel_)
107     return gpr, scaler, delaunay
108
109     df = pd.read_excel(File)
110 df = df[df['IDT'] < 1].copy()
111
112 mins = df[cols].min()
113 maxs = df[cols].max()
114
115 zones = {
116     "zone1": df[df['P'] <= 20],
117     "zone2": df[df['P'] > 20]
118 }
119
120 interpolators = {}
121 DelaunayZones = {}
122
123 for Z, subset in zones.items():
124     Xn = (subset[cols] - mins) / (maxs - mins)
125     Yn = np.log(subset['IDT'])
126
127     if Z == "zone2":
128         interpolators[Z] = inter.RBFInterpolator(
129             Xn.values, Yn.values,
130             kernel='cubic',
131             smoothing=1e-10
132         )
133
134     DelaunayZones[Z] = Delaunay(Xn.values)
135
136 gpr, scaler, delaunay = Init_GPR(File)
137
138 # Final Interpolator
139
140 def Interpolator(P, T, Fr, Y02):
141
142     # Normalisation
143     Pn = (P - mins['P']) / (maxs['P'] - mins['P'])
```

```
144     Tn = (T - mins['T']) / (maxs['T'] - mins['T'])
145     Frn = (Fr - mins['Fr']) / (maxs['Fr'] - mins['Fr'])
146     Y02n = (Y02 - mins['Y02']) / (maxs['Y02'] - mins['Y02'])
147
148     Xn = np.array([[Pn, Tn, Frn, Y02n]])
149
150     # Test GPR
151     X_std = scaler.transform([[P, T, Fr, Y02]])
152     if DelaunayZones['zone1'].find_simplex(X_std) >= 0:
153         log_idt = gpr.predict(X_std)[0]
154         return float(np.exp(log_idt))
155
156     # RBF
157     log_idt = interpolators["zone2"](Xn)[0]
158     return float(np.exp(log_idt))
```

Finally, this last code shows the training of the neural network. The different classes used have previously been defined in the code. This section only shows the neural network training process. The optuna algorithm that I decided to not show for practical reasons, is the one that gave the values for the parameters, such as the learning rate "lr" or the hidden layers sizes.

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 from torch.utils.data import DataLoader, TensorDataset
5 from sklearn.metrics import r2_score, mean_absolute_error
6 from DataProcess import make_data
7 import numpy as np
8 import joblib
9 import matplotlib.pyplot as plt
10 from sklearn.cluster import KMeans
11 from sklearn.model_selection import GroupKFold
12
13 # Parameters
14
15 folder_path = "IC8H18.xlsx"
16 lr = 0.0012981056244724973
17 weight_decay = 1e-5
18 num_epochs = 100
19 hidden_size1 = 165
20 hidden_size2 = 80
21 batch_size = 32
22 optimizer_choice = "AdamW"
23 k_folds = 3
24 seed = 1998
25
26 def train_and_save_model():
27     X_raw, Y_raw = make_data(folder_path)
28     print("Shape inputs:", X_raw.shape, "Shape outputs:", Y_raw.shape)
29
30     input_size = X_raw.shape[1]
31     output_size = Y_raw.shape[1]
32
33     params = {
34         "input_size": input_size,
35         "hidden_size1": hidden_size1,
36         "hidden_size2": hidden_size2,
37         "output_size": output_size
38     }
39     joblib.dump(params, "model_params.pkl")
40
41     kmeans = KMeans(n_clusters=k_folds, random_state=seed)
42     cluster_labels = kmeans.fit_predict(X_raw)
43
44     kfold = GroupKFold(n_splits=k_folds)
45
46     all_val_mae, all_val_mre, all_val_r2, all_val_acc = [], [], [], []
47
```

```
48
49
50 for fold, (train_idx, val_idx) in enumerate(kfold.split(X_raw, Y_raw,
51 groups=cluster_labels)):
52     print(f"\n----- Fold {fold+1}/{k_folds} -----")
53
54     X_train_raw, X_val_raw = X_raw[train_idx], X_raw[val_idx]
55     Y_train_raw, Y_val_raw = Y_raw[train_idx], Y_raw[val_idx]
56
57     input_scaler = StandardScaler().fit(X_train_raw)
58     X_train = input_scaler.transform(X_train_raw)
59     X_val = input_scaler.transform(X_val_raw)
60     Y_train = Y_train_raw
61     Y_val = Y_val_raw
62
63     X_train_t = to_tensor(X_train)
64     Y_train_t = to_tensor(Y_train)
65     X_val_t = to_tensor(X_val)
66     Y_val_t = to_tensor(Y_val)
67
68     train_ds = TensorDataset(X_train_t, Y_train_t)
69     train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle
70                               =True)
71
72     model = NeuralNetwork(input_size, hidden_size1, hidden_size2,
73                             output_size)
74     optimizer = optim.AdamW(model.parameters(), lr=lr, weight_decay=
75                               weight_decay)
76     criterion = mean_relative_error_torch
77
78     early_stopping = EarlyStopping(patience=10, min_delta=1e-4)
79
80     # Training
81
82     train_loss_per_epoch = []
83     val_loss_per_epoch = []
84     for epoch in range(1, num_epochs + 1):
85         model.train()
86         train_losses = []
87
88         for bx, by in train_loader:
89             optimizer.zero_grad()
90             preds = model(bx)
91             loss = criterion(preds, by)
92             loss.backward()
93             optimizer.step()
94             train_losses.append(loss.item())
95
96         # Validation
97
98         model.eval()
99         with torch.no_grad():
```

```

96         preds_val_t = model(X_val_t)
97         val_loss = criterion(preds_val_t, Y_val_t).item()
98
99         preds_val = preds_val_t.cpu().numpy()
100
101     train_loss_per_epoch.append(np.mean(train_losses))
102     val_loss_per_epoch.append(val_loss)
103
104     val_mae = mean_absolute_error(Y_val, preds_val)
105     val_mre = mean_relative_error(Y_val, preds_val)
106     val_r2 = r2_score(Y_val, preds_val)
107     val_acc_rel = calculate_regression_accuracy(Y_val, preds_val,
108         mode="relative", tolerance=0.1)
109
110     all_val_mae.append(val_mae)
111     all_val_mre.append(val_mre)
112     all_val_r2.append(val_r2)
113     all_val_acc.append(val_acc_rel)
114
115     early_stopping(val_loss, model)
116
117     if early_stopping.should_stop:
118         print(f"Early stopping at epoch {epoch}")
119         model.load_state_dict(early_stopping.best_state)
120         break
121
122     print("\n Average")
123     print(f"MAE : {np.mean(all_val_mae):.4f}")
124     print(f"MRE : {np.mean(all_val_mre):.4f}")
125     print(f"R2 : {np.mean(all_val_r2):.4f}")
126     print(f"Acc 10% : {np.mean(all_val_acc):.2f}%")
127
128     # Final Training
129
130     input_scaler = StandardScaler().fit(X_raw)
131     X_full = input_scaler.transform(X_raw)
132     Y_full = Y_raw
133
134     full_ds = TensorDataset(torch.tensor(X_full, dtype=torch.float32),
135                             torch.tensor(Y_full, dtype=torch.float32))
136     full_loader = DataLoader(full_ds, batch_size=batch_size, shuffle=True)
137
138     final_model = NeuralNetwork(input_size, hidden_size1, hidden_size2,
139                                 output_size)
140     optimizer = optim.AdamW(final_model.parameters(), lr=lr, weight_decay=
141                             weight_decay)
142     criterion = mean_relative_error_torch
143
144     for epoch in range(1, num_epochs + 1):
145         final_model.train()
146         for bx, by in full_loader:
147             optimizer.zero_grad()
```

```
145         preds = final_model(bx)
146         loss = criterion(preds, by)
147         loss.backward()
148         optimizer.step()
149
150     torch.save(final_model.state_dict(), "model.pth")
151     joblib.dump(input_scaler, "input_scaler.pkl")
152
153     # Final evaluation
154
155     final_model.eval()
156     with torch.no_grad():
157         preds_final = final_model(torch.tensor(X_full, dtype=torch.float32
158         )).numpy()
159
160     final_mae = mean_absolute_error(Y_full, preds_final)
161     final_mre = mean_relative_error(Y_full, preds_final)
162     final_r2 = r2_score(Y_full, preds_final)
163
164     print("\n Final stats ")
165     print(f"final MAE : {final_mae:.4f}")
166     print(f"final MRE : {final_mre:.4f}")
167     print(f"final R2 : {final_r2:.4f}")
168
169     return final_model, input_scaler
```

Isae supméca

Institut supérieur de mécanique de Paris
3 rue Fernand Hainaut 93407 Saint-Ouen
Cedex tél. 01 49 45 29 00 / www.isae-supmeca.fr

Ministère de l'Enseignement
supérieur et de la Recherche

GROUPE
Isae 